

Mini-Git -- Project Report

GitHub Repository

https://github.com/huanghei1997/Mini_Git_Test_V0.1

Overview

Mini-Git is a lightweight version control system built with C++17, featuring an interactive terminal UI (TUI). It supports file tracking, snapshot-based commits, branching, quick save/restore, and a commit tree visualization.

Features

- **File Tracking** -- Add/remove files via an in-app file browser with multi-select
- **Commit Management** -- Full-snapshot commits preserving complete file contents
- **Branch Operations** -- Create branches from any commit, switch between branches, delete branches
- **Quick Save** -- Save/Restore temporary working state without committing
- **Commit Tree View** -- Visualize multi-branch commit history with interactive navigation
- **Terminal UI** -- Fully keyboard-driven interface built with FTXUI

Technology Stack

Component	Choice
Language	C++17
Build System	CMake 3.14+
TUI Framework	FTXUI v5.0
Serialization	nlohmann/json
File Operations	std::filesystem
Platform	Windows / Linux (cross-platform)

Build & Run

```
# Build
mkdir build && cd build
cmake ..
cmake --build .

# Run TUI
./mini-git

# Run tests
./mini-git --test
```

Keyboard Shortcuts

Key	Action	Context
Q	Quit	Main view
F	File list	Global
A	Open file browser (multi-select)	File list
D	Untrack file / Delete branch	File list / Tree view
C	Create commit	Global

Key	Action	Context
T	Commit tree (branch + version management)	Global
B	Create branch from selected node	Tree view
S	Quick save	Global
R	Restore save	Global
P	Promote save to commit	Global
?	Help panel	Global
Esc	Go back	Sub-views
Arrow keys	Navigate	Lists / Tree / Browser
Space	Toggle file selection	File browser
Enter	Enter directory / Confirm / Checkout	Browser / Dialog / Tree
Backspace	Go to parent directory	File browser

Project Structure

----	CMakeLists.txt	
----	README.md	
----	include/	
----	core/	
----	repository.h	# Repository structure & initialization
----	file_tracker.h	# File tracking & status detection
----	commit.h	# Commit create/load/checkout
----	branch.h	# Branch management
----	save.h	# Quick save/restore
----	snapshot.h	# Snapshot engine
----	tui/	
----	app.h	# TUI entry point
----	tree_view.h	# Tree data structures
----	utils/	
----	path_utils.h	# Path normalization/encoding
----	time_utils.h	# Timestamp utilities
----	src/	
----	main.cpp	# Entry point + integration tests
----	core/	# Backend implementation
----	tui/	# TUI implementation
----	.gitignore	

Design Decisions

Decision	Choice	Rationale
Snapshot strategy	Full copy	Simple implementation, acceptable disk usage for small projects
Commit ID	Zero-padded 3-digit number	Simple, intuitive, directory-sort friendly
Data format	JSON (nlohmann/json)	Human-readable, easy to debug
UI framework	FTXUI	Cross-platform terminal UI, no external dependencies
Path handling	std::filesystem	C++17 standard, cross-platform

Architecture

The system follows a layered architecture:

- 1
- 2
- Core Layer** (src/core/) -- Backend logic: repository, commits, branches, saves, snapshots
 - TUI Layer** (src/tui/) -- Terminal interface: views, navigation, event handling

3. **Utils Layer** (`src/Utils/`) -- Cross-platform path and time utilities

Data is stored in `.minigit/` under the project root:

```
.minigit/
|---- config          # Current branch name (JSON)
|---- tracked_files   # List of tracked file paths (JSON)
|---- branches/
|---- main/
|---- commits/
|    |---- 001/
|    |    |---- meta.json    # Commit metadata (id, message, timestamp, parent, files)
|    |    |---- files/      # Snapshot of all tracked files
|    |---- 002/
|    |---- ...
|---- save/
|---- files/          # Quick save snapshot
```

Testing

The project includes comprehensive tests covering all phases:

- **Phase 1 tests** -- Path utils, repository init, file tracking, commits, branches
- **Phase 2 tests** -- Save create/restore/promote
- **E2E integration** -- Full workflow: track -> commit -> branch -> switch -> save -> restore -> promote -> checkout -> untrack

All tests pass via `./mini-git --test`.

Source Code

Complete source code (27 files, ~2700 lines) is available in: - **GitHub**:
https://github.com/huanghei1997/Mini_Git_Test_V0.1 - **Archive**: `all_source.cpp` (single-file reading copy)